

## **Estructuras de datos en JavaScript:**

Arreglos

Objetos

Métodos de arreglo (push, pop, shift, filter, etc.)

Desestructuración.

## Situación - El Carrito de Compras

Hasta ahora, nuestro código ha manejado datos solitarios: una variable para la edad (`let edad = 20;`), otra para el nombre (`let nombre = "Ana";`).

Imagina que estás construyendo una tienda en línea como Amazon. Cuando un usuario agrega un producto a su carrito, ¿cómo "recuerda" el sistema todo lo que ha añadido?

No es una sola cosa, es una lista. Y cada producto en esa lista no es solo un nombre; tiene un precio, una cantidad, una imagen, un ID único.

Si el usuario cambia la cantidad de un artículo o elimina otro, el total se actualiza instantáneamente.

Toda esa lógica, desde guardar la lista de productos hasta calcular el total, se gestiona con las herramientas que vamos a aprender hoy: **arreglos** y **objetos**. Son los contenedores fundamentales de la información en la web.

Sin arreglos ni objetos, es imposible construir aplicaciones web modernas. Todo API que consultes en el futuro (clima, usuarios, productos) te devolverá la información usando estas dos estructuras.

# Los Arreglos (Arrays) - Las Listas Ordenadas

Un arreglo es una estructura de datos que nos permite almacenar una colección de múltiples elementos en una sola variable. Piensa en ello como una estantería con espacios numerados.

**Sintaxis y Creación:** Se declaran con corchetes [], y los elementos se separan por comas.

// Un arreglo de strings

```
let playlist = ['Bohemian Rhapsody', 'Stairway to Heaven', 'Hotel California'];
```

// Un arreglo de números

```
let calificaciones = [10, 9, 8, 7, 10];
```

// Un arreglo mixto

```
let datosMixtos = ['Ana', 28, true, null];
```

## Acceso y Modificación

El acceso a los elementos se hace a través de su **índice**, que es su posición numérica. El primer elemento siempre está en el índice 0.

```
console.log(playlist[0]); // Muestra 'Bohemian Rhapsody'  
console.log(playlist[2]); // Muestra 'Hotel California'
```

```
// Para modificar un elemento, lo seleccionamos por su índice  
playlist[1] = 'Imagine - John Lennon';  
console.log(playlist); // ['Bohemian Rhapsody', 'Imagine - John Lennon', 'Hotel California']
```

La propiedad **.length** nos dice cuántos elementos hay. `playlist.length` devolvería 3.

Con la función `sort()`, se puede ordenar un array.

## Métodos Fundamentales (Mutadores):

**push(elemento):** Agrega uno o más elementos **al final**. Es como poner un libro más encima de una pila.

**pop():** Elimina y devuelve el **último** elemento. Es como quitar el libro de encima de la pila.

**unshift(elemento):** Agrega uno o más elementos **al inicio**. Esto es "costoso" en rendimiento porque debe re-indexar todos los demás elementos.

**shift():** Elimina y devuelve el **primer** elemento. También es "costoso" por la misma razón.

```
let frutas = ['manzana', 'banana'];  
frutas.push('naranja'); // Agrega uno o más elementos al final del arreglo.  
console.log(frutas); // Resultado: ['manzana', 'banana', 'naranja']
```

```
let colores = ['rojo', 'verde', 'azul'];  
colores.pop(); // Elimina el último elemento del arreglo.  
console.log(colores); // Resultado: ['rojo', 'verde']
```

```
let numeros = [3, 4, 5];  
numeros.unshift(1, 2); //Agrega uno o más elementos al principio del arreglo.  
console.log(numeros); // Resultado: [1, 2, 3, 4, 5]
```

```
let animales = ['perro', 'gato', 'pájaro'];  
animales.shift(); //Elimina el primer elemento del arreglo  
console.log(animales); // Resultado: ['gato', 'pájaro']
```

## Métodos Modernos (Iteradores / No Mutadores):

Estos son los más usados en el desarrollo moderno. No modifican el arreglo original, sino que **retornan un nuevo arreglo o un valor**. Son la base de la programación funcional en JavaScript.

**forEach(callback):** Ejecuta una función (el *callback*) para cada elemento del arreglo. No retorna nada (undefined). Es un reemplazo moderno del bucle for.

```
const numeros = [1, 2, 3];
numeros.forEach(function(numero, indice) {
  console.log(`El elemento ${numero} está en el índice ${indice}`);
});
```

**map(callback):** Transforma cada elemento y devuelve un nuevo arreglo con los resultados. Esencial en frameworks como React para crear listas de componentes.

**Analogía:** Imagina una máquina que toma una lista de naranjas ([naranja, naranja]) y te devuelve una lista de jugos ([jugo, jugo]). El arreglo original de naranjas sigue intacto.

```
const dobles = numeros.map(function(numero) {  
  return numero * 2;  
}); // dobles es [2, 4, 6]
```

**filter(callback):** Crea un **nuevo arreglo** con todos los elementos que pasen una prueba (el *callback* debe devolver true).

**Analogía:** Es como un colador. Tienes una mezcla de arena y piedras, y el colador (filter) te deja solo con las piedras (los elementos que cumplen la condición).

```
const pares = [1, 2, 3, 4, 5, 6].filter(function(n) {  
  return n % 2 === 0; // La condición: ¿es el número par?  
}); // pares es [2, 4, 6]
```

**reduce(callback, valorInicial):** Reduce el arreglo a un único valor. Ejecuta una función "acumuladora" para cada elemento.

- Es el más potente pero también el más complejo de entender al principio.
- El callback recibe dos argumentos principales: acumulador y valorActual.

```
const carrito = [10, 25, 15, 50];  
const total = carrito.reduce(function(sumaTotal, precioActual) {  
  return sumaTotal + precioActual;  
}, 0); // El 0 es el valor inicial de sumaTotal  
// total es 100
```

**find(callback):** Devuelve el **primer elemento** que cumple la condición. Si no lo encuentra, devuelve undefined.

**some(callback):** Devuelve true si **al menos un** elemento cumple la condición.

**every(callback):** Devuelve true si **todos** los elementos cumplen la condición.

# Ejemplos

map() - Transforma cada elemento

```
let numeros = [1, 2, 3];  
let dobles = numeros.map(function(numero) {  
  return numero * 2;  
});  
console.log(dobles); // Resultado: [2, 4, 6]  
console.log(numeros); // El original no cambió: [1, 2, 3]
```

filter() - Filtra elementos

```
let edades = [15, 20, 30, 12];  
let adultos = edades.filter(function(edad) {  
  return edad >= 18;  
});  
console.log(adultos); // Resultado: [20, 30]
```

reduce() - Reduce a un solo valor

```
let carrito = [10, 25, 5];  
let total = carrito.reduce(function(suma, precioActual) {  
  return suma + precioActual;  
}, 0); // El 0 es el valor inicial de la suma
```

```
console.log(total); // Resultado: 40
```

# Los Objetos (Objects) - Los Diccionarios de Datos

Un objeto es una colección de datos relacionados, almacenados en pares de clave: valor (key-value pairs). Es ideal para describir una sola entidad en detalle.

- Sintaxis y Creación: Se declaran con llaves {}.

```
const usuario = {  
  nombre: 'Carlos Rodriguez',  
  edad: 32,  
  esEstudiante: false,  
  cursos: ['HTML', 'CSS', 'JavaScript'],  
  direccion: {  
    calle: 'Av. Siempre Viva',  
    numero: 123  
  },  
  saludar: function() {  
    console.log('¡Hola mundo!');  
  }  
};
```

# Acceso y Modificación

Tenemos dos formas de acceder a los valores:

1. **Notación de punto (.):** La más común y fácil de leer. `console.log(usuario.nombre); // 'Carlos Rodriguez'` `console.log(usuario.direccion.calle); // 'Av. Siempre Viva'`

2. **Notación de corchetes ([]):** Obligatoria cuando la clave tiene espacios, caracteres especiales o es una variable.

```
console.log(usuario['edad']); // 32
```

```
let propiedad = 'nombre';
```

```
console.log(usuario[propiedad]); // 'Carlos Rodriguez'
```

Para modificar o añadir, simplemente asignamos un valor: `usuario.edad = 33;` `usuario.username = 'carlos_rdz';` // Se añade una nueva propiedad

# Desestructuración (Destructuring) - Desempacando Valores

Es una sintaxis de JavaScript que permite "desempacar" valores de arreglos u objetos en distintas variables. Hace el código más corto y legible.

- **Desestructuración de Objetos:** Extraemos propiedades por su nombre.

```
const { nombre, edad, cursos } = usuario;  
console.log(nombre); // 'Carlos Rodriguez'  
console.log(edad); // 33
```

```
// Podemos renombrar variables  
const { nombre: nombreCompleto } = usuario;  
console.log(nombreCompleto); // 'Carlos Rodriguez'
```

```
// Podemos asignar valores por defecto  
const { username, tema = 'oscuro' } = usuario;  
console.log(tema); // 'oscuro' (porque no existe en el objeto)
```

## Desestructuración de Arreglos:

Extraemos elementos por su posición.

```
const misCalificaciones = [10, 9, 8, 7];  
const [primera, segunda] = misCalificaciones;  
console.log(primera); // 10  
console.log(segunda); // 9
```

```
// Podemos ignorar elementos con una coma vacía  
const [, tercera] = misCalificaciones;  
console.log(tercera); // 8
```

## Preguntas Frecuentes

**¿Por qué usar la notación de corchetes objeto['propiedad'] si la de punto objeto.propiedad es más fácil?**

La notación de punto es la preferida por su simplicidad, pero tiene limitaciones. Solo funciona con nombres de propiedades válidos y estáticos (que no tienen espacios, guiones, ni empiezan con números). La notación de corchetes es tu navaja suiza: te permite acceder a propiedades con nombres complejos (miObjeto['nombre-con-guiones']) y, lo más importante, te permite usar una **variable** para determinar a qué propiedad acceder. Esto es fundamental cuando el nombre de la propiedad que necesitas leer es dinámico y cambia durante la ejecución del programa.

**¿Por qué puedo agregar elementos a un arreglo con push si declaré el arreglo con const? ¿No se supone que const prohíbe los cambios?**

const prohíbe la **reasignación de memoria** del contenedor completo, pero no su mutación interna. Es decir, no puedes hacer lista = ["Otro arreglo"], pero sí puedes modificar los elementos que viven dentro de los corchetes originales. Es como tener una casa (const); no puedes cambiar la dirección de la casa, pero sí puedes agregar o quitar muebles adentro.

**¿Debería mezclar diferentes tipos de datos en un arreglo (ej. ["Texto", 42, true] )?**

Aunque JavaScript lo permite por su tipado dinámico, es una pésima práctica arquitectónica. Los arreglos deben contener elementos del mismo tipo o estructura (ej. solo strings, o solo objetos de productos) para evitar errores lógicos al aplicar métodos como filter.

## Spread Operator (...)

En React no modificamos la lista original, creamos una copia exacta, le añadimos el nuevo dato, y reemplazamos la lista vieja.

```
const mockUsuarios = ["Ana", "Carlos"];
```

```
// Forma React (Inmutable usando Spread)
```

```
const nuevosUsuarios = [...mockUsuarios, "Beatriz"];
```

```
console.log(mockUsuarios); // ["Ana", "Carlos"] (Intacto)
```

```
console.log(nuevosUsuarios); // ["Ana", "Carlos", "Beatriz"] (Nuevo estado)
```

# Combinando Arreglos y Objetos con filter()

Este es el patrón más común en desarrollo web: un arreglo que contiene objetos. Vamos a buscar productos baratos.

```
const productos = [  
  { id: 1, nombre: "Camiseta", precio: 25 },  
  { id: 2, nombre: "Zapatos", precio: 80 },  
  { id: 3, nombre: "Gorra", precio: 15 },  
  { id: 4, nombre: "Chaqueta", precio: 120 }  
];  
  
// Queremos solo los productos que cuesten menos de 50  
// .filter() ejecuta la función flecha por cada producto.  
// Si la flecha retorna 'true', el producto se guarda en la nueva lista.  
const ofertas = productos.filter(producto => producto.precio < 50);  
  
console.log("Productos en oferta:", ofertas);  
// Resultado: [{ id: 1, nombre: "Camiseta", precio: 25 }, { id: 3, nombre: "Gorra", precio: 15 }]
```

# Ejemplos demostrativos

Vamos a simular el núcleo de un carrito de compras combinando Arreglos y Objetos, y aplicaremos métodos para gestionar la lógica de negocio.

```
"use strict";
```

```
// 1. Declaramos una estructura de datos compleja: un Arreglo de Objetos
const carritoCompras = [
  { id: 1, producto: "Laptop", precio: 1200, categoria: "Tecnología" },
  { id: 2, producto: "Libro JavaScript", precio: 40, categoria: "Educación" }
];
```

```
// 2. Uso de Push: El usuario agrega un nuevo producto al carrito
const nuevoProducto = { id: 3, producto: "Mouse Inalámbrico", precio: 25, categoria: "Tecnología" };
carritoCompras.push(nuevoProducto);
console.log("Carrito actualizado:", carritoCompras);
```

```
// 3. Uso de Filter: El usuario quiere ver solo los productos de 'Tecnología'
const productosTecnología = carritoCompras.filter(item => item.categoria === "Tecnología");
console.log("Filtrado por Tecnología:", productosTecnología);
```

```
// 4. Destructuración: Extraemos datos del primer producto para mostrar un banner promocional
const primerItem = carritoCompras[0];
const { producto, precio } = primerItem;
```

```
console.log(`¡Lleva hoy la ${producto} por solo ${precio}!`);
```

# Práctica de Afianzamiento: La Fila del Banco

Simular la gestión de clientes usando métodos de arreglo.

Paso a paso:

1. Crea un arreglo llamado filaBanco con 3 nombres: "Ana", "Carlos", "Beatriz".
2. Un nuevo cliente llega a la fila: "David". Agrégalo al **final** de la fila.
3. Otro cliente tiene urgencia y lo atienden de inmediato, colándose al **principio**: "Elena". Agrégala al principio.
4. El cajero llama al primer cliente. Retíralo de la fila y guarda su nombre en una variable atendido.
5. Imprime el nombre del cliente atendido y cómo quedó la fila final.

## Actividad de clase

**Driver:** Toma el teclado, redacta la sintaxis estructural.

**Navigator:** Lee las instrucciones, revisa nombres de propiedades y uso de métodos.

Creen un arreglo llamado `pacientesSalaEspera` que contenga tres nombres de tipo `String` (ej. "Carlos", "Ana", "Luis").

Un nuevo paciente llamado "Sofía" llega con emergencia. Utilicen el método correcto para ingresarla de **primera** en la lista.

El médico llama al primer paciente. Utilicen el método correcto para extraer al primero de la lista y guárdenlo en una variable llamada `pacienteAtendido`.

Impriman en consola: "El paciente atendido es: [`pacienteAtendido`]"

Luego impriman el estado actual del arreglo `pacientesSalaEspera`.

# Práctica Integradora

## Ticket JIRA-117: Gestión de Pila de Correos

**A. Instrucciones:** Simula una bandeja de entrada. Recibes un correo nuevo y debes agregarlo al principio de la lista. Luego, el sistema elimina el correo más antiguo (el del final de la lista) por falta de espacio. Imprime la bandeja final.

### B. Código Base:

```
const bandejaEntrada = ["Correo 2 (Ayer)", "Correo 1 (Hace una semana)"];
const nuevoCorreo = "Correo 3 (Hoy)";
// Implementa la lógica de entrada y salida aquí
```

### C. Solución Completa:

```
const bandejaEntrada = ["Correo 2 (Ayer)", "Correo 1 (Hace una semana)"];
const nuevoCorreo = "Correo 3 (Hoy)";
```

```
// Agregar al inicio
bandejaEntrada.unshift(nuevoCorreo);
```

```
// Eliminar el último
bandejaEntrada.pop();
```

```
console.log("Bandeja actualizada:", bandejaEntrada);
```

# Ticket JIRA-118: Limpieza de Base de Datos de Usuarios

**A. Instrucciones:** Tienes un arreglo de objetos de usuarios. El equipo de seguridad requiere una nueva lista que contenga **solo** a los usuarios que tengan la propiedad `cuentaActiva` en `true`.

## B. Código Base:

```
const usuarios = [  
  { id: 101, username: "dev_santiago", cuentaActiva: true },  
  { id: 102, username: "admin_test", cuentaActiva: false },  
  { id: 103, username: "user_89", cuentaActiva: true }  
];  
// Implementa el filtrado aquí
```

## C. Solución Completa:

```
const usuarios = [  
  { id: 101, username: "dev_santiago", cuentaActiva: true },  
  { id: 102, username: "admin_test", cuentaActiva: false },  
  { id: 103, username: "user_89", cuentaActiva: true }  
];  
  
const usuariosPermitidos = usuarios.filter(usuario => usuario.cuentaActiva === true);  
console.log("Usuarios activos:", usuariosPermitidos);
```

# Ticket JIRA-119: Formateador de Tarjetas de Perfil

**A. Instrucciones:** Recibes un objeto masivo desde una API simulada. Desestructura **únicamente** el nombre y el cargo del objeto. Luego, imprime un mensaje que diga: "Perfil: [nombre] - Cargo: [cargo]".

## B. Código Base:

```
const apiResponse = {  
  id: 994,  
  nombre: "Laura Rojas",  
  cargo: "Tech Lead",  
  departamento: "Ingeniería",  
  fechaIngreso: "2021-05-10"  
};  
// Desestructura e imprime aquí
```

## C. Solución Completa:

```
const apiResponse = {  
  id: 994,  
  nombre: "Laura Rojas",  
  cargo: "Tech Lead",  
  departamento: "Ingeniería",  
  fechaIngreso: "2021-05-10"  
};  
  
const { nombre, cargo } = apiResponse;  
console.log(`Perfil: ${nombre} - Cargo: ${cargo}`);
```

## Ejercicio independiente aplicado - Instrucciones (Ticket JIRA-211 - Inventario de Biblioteca Web):

El cliente necesita que inicies la estructura de datos para el módulo de biblioteca del sistema en la rama feature/inventario-libros.

1. Crea un arreglo llamado `inventarioLibros` que contenga tres objetos. Cada objeto debe representar un libro con tres propiedades: `titulo` (String), `autor` (String) y `disponible` (Booleano).
2. El sistema adquiere un nuevo libro. Utiliza un método de arreglo para insertar un nuevo objeto de libro al **final** del inventario.
3. El sistema requiere mostrar a los estudiantes solo los libros que pueden llevarse. Utiliza el método `filter` para crear un nuevo arreglo llamado `librosDisponibles` que contenga solo los libros donde `disponible` sea `true`.
4. Elige el primer libro del arreglo `librosDisponibles` y utiliza **Desestructuración** para extraer su `titulo` y `autor`. Imprime en consola un mensaje de recomendación usando esas variables.

### Criterios de logro esperados:

- Sintaxis correcta de llaves y corchetes (anidación de objetos dentro de un arreglo).
- Uso inmutable del método `filter` almacenando su retorno en la constante `librosDisponibles`.
- Desestructuración aplicada correctamente sin arrojar `undefined`.
- Integración mediante PR en GitHub sin errores de linting.